Math 116 Paper - Elliptic Curve Cryptography

Tony Angel Villalpando

May 9, 2025

Contents

1 Introduction **2**

2 Elliptic Curves **2**

3 Coding Addition of an Elliptic Curve **4**

4 Elliptic Curves over Finite Fields and the Discrete Logarithm Problem **6**

5 References **8**

1 Introduction

Why would we want to use elliptic curves in cryptography? Elliptic curves provide an alternate form of encrypting information. There have been a wide variety of methods we have seen in this class for keeping information secure, such as the Diffie-Hellman Key Exchange or the ElGamal public key cryptosystem. Elliptic curves let us use these methods in different ways, allowing for a more complex system of algebra that keeps information secure. Cryptanalysis has already been done extensively on elliptic curve cryptography systems for years, namely those surrounding problems such as the Discrete Logarithm Problem, but learning these methods helps us gain insight into the many types of encryption that exist in the cryptographical world, as well as allowing us to venture into familiar territory, algebra!

2 Elliptic Curves

An elliptic curve is the set of solutions to the Weierstrass equations of the form $y^2 = x^3 + Ax + B$. One of the restrictions that we place on these curves is for $4A^3 + 27B^2 \neq 0$. This is to ensure that our elliptic curve has certain properties that make algebra significantly more manageable.

An example of an elliptic curve is shown below:

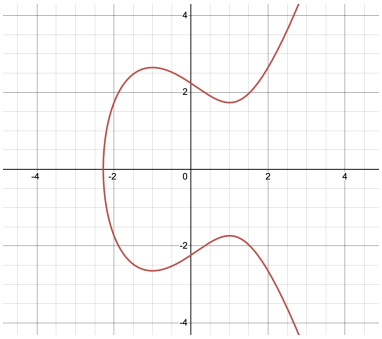


Figure 1: $y^2 = x^3 - 3x + 5$

You will notice that the curve is symmetric about the x-axis. This is yet another property of elliptic curves that makes algebra easier to work with. However, what makes these curves so special? One reason is our ability to perform "addition" on them.

Consider the elliptic curve $E : y^2 = x^3 - 4x + 4$.. We can then choose two points on this curve, namely P and Q. Let $P = (-2, 2)$ and $Q = (0, 2)$. Geometrically, we would define the addition of P and Q as follows: Take a secant line between P and Q. This line should then intersect with some other point in E, which we will call R. We can then reflect this point across the x-axis, resulting in R'. Thus, we would say $P \oplus Q = R'$.

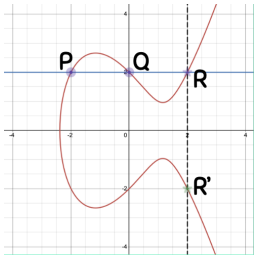


Figure 2: $P \oplus Q = R'$ in $E : y^2 = x^3 - 4x + 4$

Algebraically, this can first be done by finding the secant line given two points, (a, b) and (c, d). To do so, we would take the slope to be $\frac{d-b}{c-a} = \frac{2-2}{0-(-2)} = 0$. Then, to find the y-intercept of the secant line, we would simply compute $b - slope \cdot a = 2 - 0(-2) = 2$. Thus, the equation of the secant line is given by $y = 2$. To find the third point at which this line intersects E, we could simply plug this equation back into $E \Rightarrow 2^2 = x^3 - 4x + 4$. This should yield three solutions, two of which we already know to be a and c, or 0 and -2. Our third x-value turns out to be 2. Plug this back into our secant line equation, and we find that our point $R = (2, 2)$. The last step is to find R', which can be done by reflecting the point across the x-axis. Thus, we find that $P \oplus Q = R'$, or we can write it as $(-2, 2) \oplus (0, 2) = (2, -2)$.

Now that we have defined addition of points, we can define what it means to add a point to itself. This can be written as $P \oplus P$ or $2P$. To do this, we simply take the tangent line at the point P. This tangent line should intersect E elsewhere, and we can repeat the same process as before.

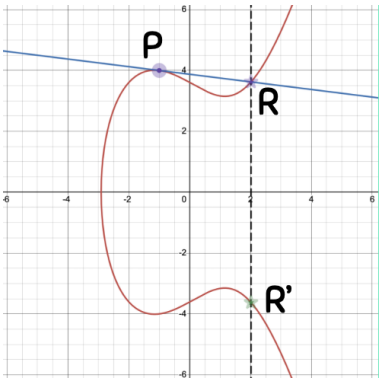


Figure 3: $2P = R'$ in E

Algebraically, we only change the way in which we find the slope. This can be done using a method from multivariable calculus. To find the slope at the point P, we simply find the negative partial derivative of E with respect to y and divide by the positive partial derivative of E with respect to x. Thus, the slope of the tangent line can be found by plugging $P = (a, b)$ into the equation $\frac{3a^2+A}{2b}$, where $E : y^2 = x^3 + Ax + B$.

Given a point, $P = (a, b)$, we define its inverse to be $P' = (a, -b)$. This is because adding these two points results in a perfectly vertical line, thus resulting in a non-existent point at infinity which we denote by Θ. Thus, we say that $P \oplus P' = \Theta$.

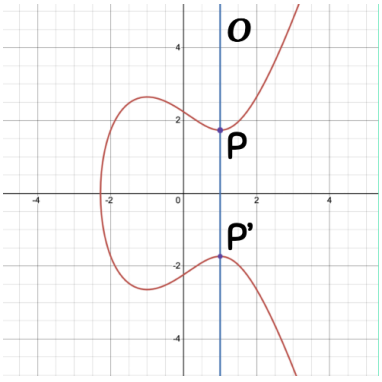


Figure 4: $P \oplus P' = \Theta$ in E

From the previous information, we are also able to derive that given any point, P, in E, $P \oplus \Theta = P$. This makes Θ an additive identity. Together with all the previous information we have defined, addition over an elliptic curve forms an abelian group.

Given any points $P, Q \in E$, $P \oplus Q \in E$, thus E has closure. Given any points $P, Q, R \in E$, $(P \oplus Q) \oplus R = P \oplus (Q \oplus R)$, thus addition in E is associative. Given any point $P \in E$, $P \oplus \Theta = \Theta \oplus P = P$, thus E has an additive identity. Given any point $P \in E$, there exists some $P' \in E$ such that $P \oplus P' = P' \oplus P = \Theta$, thus every point in E has an inverse. Lastly, given any $P, Q \in E$, $P \oplus Q = Q \oplus P$, since we are simply taking the secant line between points. Thus, addition is commutative in E.

3 Coding Addition of an Elliptic Curve

```python
def poly_div(root, poly):
    seq = []
    while len(root) <= len(poly):
        if poly[0] == 0:
            seq += [0]
            poly = poly[1:]
        else:
            value = poly[0] / root[0]
            seq += [value]
            poly[1] = poly[1] + root[1] * (-value)
            poly = poly[1:]
    return seq


class EllipticCurve:

    def __init__(self, A, B):
        assert 4*(A**3) + 27*(B**2) != 0
        self.A = A
        self.B = B

    def is_in_curve(self, p):
        val = (p.x ** 3) + (self.A * p.x) + self.B
        if val == (p.y ** 2):
            return True
        else:
            return False
```

```python
def add(self, p1, p2):
    if p1.x == p2.x and p1.y == p2.y:
        slope = (3*(p1.x ** 2) + self.A) / (2 * p1.y)
        y_int = p1.y - slope * p1.x
        poly = [1, -(slope ** 2), (2 * slope * y_int) + self.A, self.B - (y_int ** 2)]
        root = [1, -p1.x]
        poly_2 = poly_div(root, poly)
        poly_3 = poly_div(root, poly_2)
        new_x = -poly_3[1]
        r = Point(new_x, -(slope * new_x + y_int))
        return r
    elif p1 is Omega:
        return p2
    elif p2 is Omega:
        return p1
    elif p1.x == p2.x and p1.y == - p2.y:
        return Omega
    else:
        slope = (p2.y - p1.y) / (p2.x - p1.x)
        y_int = p1.y - (slope * p1.x)
        poly = [1, -(slope ** 2), (2 * slope * y_int) + self.A, self.B - (y_int ** 2)]
        root_1 = [1, -p1.x]
        root_2 = [1, -p2.x]
        poly_2 = poly_div(root_1, poly)
        poly_3 = poly_div(root_2, poly_2)
        new_x = -poly_3[1]
        r = Point(new_x, -(slope * new_x + y_int))
        return r

def add_self(self, p):
    slope = (3*(p.x ** 2) + self.A) / (2 * p.y)
    y_int = p.y - slope * p.x
    poly = [1, -(slope ** 2), (2 * slope * y_int) + self.A, self.B - (y_int ** 2)]
    root = [1, -p.x]
    poly_2 = poly_div(root, poly)
    poly_3 = poly_div(root, poly_2)
    new_x = -poly_3[1]
    r = Point(new_x, -(slope * new_x + y_int))
    return r
```

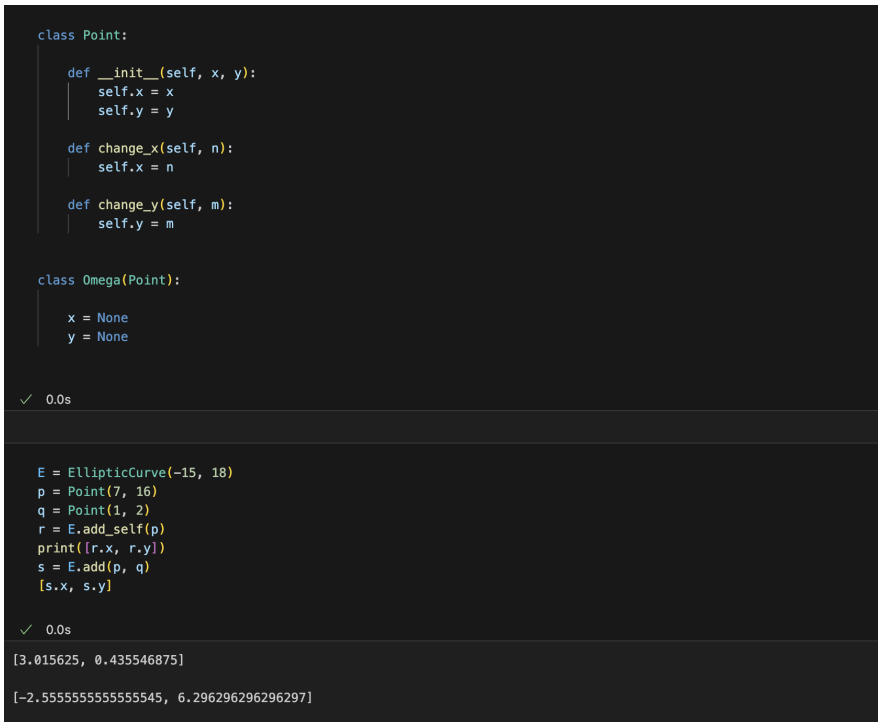


Figure 5: Using classes in Python to compute the addition of points in E

4 Elliptic Curves over Finite Fields and the Discrete Logarithm Problem

We now turn to elliptic curves over some finite field of order p, which we denote by $E(\mathbb{F}_p)$. In an elliptic curve over a finite field, only a finite set of points exist. Consider the elliptic curve $E : y^2 = x^3 + 3x + 8$ over $\mathbb{F}_{13}$. We can plug in x-values between 0 and 12 to find which exist. For instance, if we plug in 1, we are left with the equation $y^2 = 12$. This tells us that the x-value 1 only exists in $E(\mathbb{F}_{13})$ if 12 is a quadratic residue of 13. This means we want to see if the solution $y^2 \equiv 12 \mod 13$ has a solution. This equations holds when $y \equiv 5, 8$, thus the points $(1, 5)$ and $(1, 8)$ exist in $E(\mathbb{F}_{13})$.

If we plugged in all the values between 0 and 12 to see which were quadratic residues, we would find that the points that exist in $E(\mathbb{F}_{13})$ are $\{(1, 5), (1, 8), (2, 3), (2, 10), (9, 6), (9, 7), (12, 2), (12, 11), \Theta\}$. Thus, $E(\mathbb{F}_{13})$ is an abelian group of order 9.

Since quadratic residues appear 50% of the time, we can generally expect the group $E(\mathbb{F}_p)$ to have an order of around $p + 1$.

This leads to a few problems. If we know that elliptic curves over a finite field result in a finite abelian group, then we know that any given point in the group must have a finite order. This leads to the formation of the equation $nP = \Theta$, which means we want to find how many times we have to add P to itself to obtain the identity of the group. This can be done with a variety of methods, but is most quickly done with a coding program.

If we consider an elliptic curve over a finite field of large prime order, this computation can become significantly harder. Consider $E : y^2 = x^3 + 8x + 7$ over $\mathbb{F}_{73}$. If we were to find all the points that exist in this group, we would find the order to be 82. However, this computation takes too long, thus, we focus instead on finding the order of a given point in the group. If we take $P = (32, 53)$, we'll notice that we must add it to itself 41 times to get to the identity. Thus, $41P = \Theta$, which means the order of P is 41 (which you'll notice divides 82.

This leads directly into the discrete logarithm problem, which systems like Elliptic Diffie-Hellman key exchange or Elliptic ElGamal cryptosystems rely on. These are equations of the form $nP = Q$, where P and Q are two points in $E(\mathbb{F}_p)$. This computation can get messy, but with the right program, it can be made significantly easier!

```python
class EllipticCurveOverField:

    def __init__(self, A, B, prime):
        assert 4*(A**3) + 27*(B**2) != 0
        assert prime >= 3
        self.A = A
        self.B = B
        self.prime = prime

    def is_in_curve(self, p):
        value = ((p.x ** 3) + (self.A * p.x) + self.B) % self.prime
        legendre = (value**((self.prime - 1) / 2)) % self.prime
        if legendre == 1:
            return True
        else:
            return False

    def add(self, p1, p2):
        if p1.x == p2.x and p1.y == p2.y:
            value = 2 * p1.y
            for i in range(self.prime):
                if ((value * i) % self.prime) == 1:
                    mult_inv = i
                    break
            slope = ((3*(p1.x ** 2) + self.A) * mult_inv) % self.prime
            y_int = (p1.y - slope * p1.x) % self.prime
            new_x = ((slope ** 2) - (2 * p1.x)) % self.prime
            new_y = (-((slope * new_x) + y_int)) % self.prime
            r = Point(new_x, new_y)
            return r
        elif p1 is Omega:
            return p2
        elif p2 is Omega:
            return p1
        elif p1.x == p2.x and p1.y == (-p2.y) % self.prime:
            return Omega
        else:
            value = p2.x - p1.x
            for i in range(self.prime):
                if ((value * i) % self.prime) == 1:
                    mult_inv = i
                    break
```

```
            slope = ((p2.y - p1.y) * mult_inv) % self.prime
            y_int = (p1.y - slope * p1.x) % self.prime
            new_x = ((slope ** 2) - p1.x - p2.x) % self.prime
            new_y = (-((slope * new_x) + y_int)) % self.prime
            r = Point(new_x, new_y)
            return r

    def add_self(self, p):
        value = 2 * p.y
        for i in range(self.prime):
            if ((value * i) % self.prime) == 1:
                mult_inv = i
                break
        slope = ((3*(p.x ** 2) + self.A) * mult_inv) % self.prime
        y_int = (p.y - slope * p.x) % self.prime
        new_x = ((slope ** 2) - (2 * p.x)) % self.prime
        new_y = (-((slope * new_x) + y_int)) % self.prime
        r = Point(new_x, new_y)
        return r

    def order(self, p):
        r = p
        counter = 1
        while r is not Omega:
            q = E.add(r, p)
            r = q
            counter += 1
        return counter

    def logarithm(self, p, q):
        r = p
        value = E.order(p)
        counter = 1
        while r.x != q.x or r.y != q.y:
            s = E.add(r, p)
            r = s
            counter += 1
            print([r.x, r.y])
            if counter > value:
                return None
        return counter
```

```
E = EllipticCurveOverField(8, 7, 73)
p = Point(32, 53)
E.order(p)
```

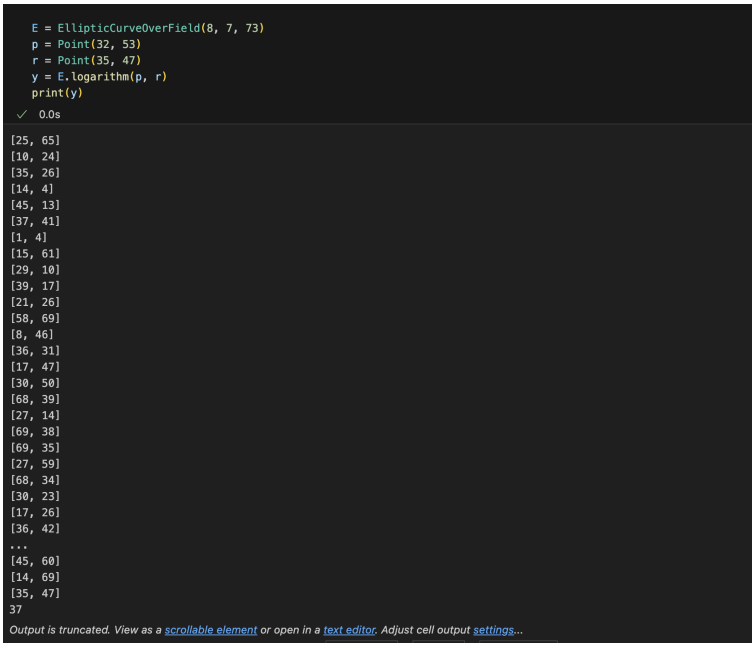
✓ 0.0s

```
41
```

```
E = EllipticCurveOverField(8, 7, 73)
p = Point(32, 53)
q = Point(39, 17)
x = E.logarithm(p, q)
print(x)
```

✓ 0.0s

```
[25, 65]
[10, 24]
[35, 26]
[14, 4]
[45, 13]
[37, 41]
[1, 4]
[15, 61]
[29, 10]
[39, 17]
11
```



5 References

1. Cryptanalysis Using Elliptic Curves : https://www.sciencedirect.com/science/article/pii/S0030402618305618
2. Elliptic Curve Cryptography and Machine Learning : https://www.mdpi.com/2410-387X/9/1/3
3. An Introduction to Mathematical Cryptography by Jeffrey Hoffstein, Jill Pipher, and Joseph H. Silverman (Chpt. 5)
4. A Graduate Course in Applied Cryptography by Dan Boneh and Victor Shoup (Chpt. 15)